Summer Training Report

On

Quantum Neural Networks

At



IISER Bhopal

Submitted by

Abhideep Jain

jabhideep9@gmail.com

2k23/EP/04

Fourth Semester, B.TECH in Engineering Physics

Delhi Technological University

Advisor: Dr. Kuntal Roy, EECS Department, IISER Bhopal

Email: kuntal@iiserb.ac.in

CONTENT

Table of Contents

1. **Quantum Computing**
   - 1.1 Basic Principles of Quantum Computing
   - 1.2 Quantum computing vs Classical Computing
   - 1.3 Quantum Circuits
   - 1.4 Quantum Algorithms

2. **Neural Networks**
   - 2.1 Multiple Layer Neural Network
3. **Quantum Neural Network**
   - 3.1 Feature Mapping
   - 3.2 Fisher Information Matrix
   - 3.3 Effective Dimension
4. **Classical V/s Quantum Neural Network**
   - 4.1 Fisher Information Spectrum
   - 4.2 Trainability
5. **Conclusion**
6. **References**

List of Figures

Figure 1. The circuit encoding the equation $CNOT_{12}(H \otimes I)|00 \geq \frac{1}{\sqrt{2}}(|00> + |11>)$

Figure 2. Shor's Algorithm for N=15 and x =11

Figure 3. Single Layer Neural Networks

Figure 4. Examples of handwritten digits from the MNIST corpus. Each grayscale image has 28x28 pixels, each of which is an eight-bit number (0–255) which represents how dark that pixel is. The first 3, 5, and 8 are enlarged to show their 784 individual pixel values

Figure 5. Neural network diagram with two hidden layers and multiple outputs, suitable for the MNIST handwritten-digit problem. The input layer has p =784 units, the two hidden layers $K_1 = 256$ and $K_2 = 128$ units respectively, and the output layer 10 units. Along with intercepts (referred to as biases in the deep-learning community) this network has 235,146 parameters (referred to as weights).

Figure 6. Overview of a Quantum Neural Network.

Figure 7. Feature Mapping of Quantum Neural Network

Figure 8. Box Plots of Average Eigenvalue Distribution of Fisher Information Matrix with Classical Neural Network And Two different quantum encodings

Figure 9. The normalized effective dimension plotted for the quantum neural network (green), the easy quantum model (blue) and the classical feedforward neural network (purple).

1. Quantum Computing

Quantum computing is an emergent field of computer science and engineering that harnesses the unique qualities of quantum mechanics to solve problems beyond the ability of even the most powerful classical computers. The field of quantum computing includes a range of disciplines, including quantum hardware and quantum algorithms. While still in development, quantum technology will soon be able to solve complex problems that classical supercomputers can't solve (or can't solve fast enough). By taking advantage of quantum physics, large-scale quantum computers would be able to tackle certain complex problems many times faster than modern classical machines. With a quantum computer, some problems that might take a classical computer thousands of years to solve might be solved in a matter of minutes or hours.

1.1. Basic Principles of Quantum Computing

Quantum computers take advantage of quantum mechanics by replacing traditional binary bit circuits with quantum particles called quantum bits, or qubits. These particles behave differently from bits, exhibiting unique properties that can be described only with quantum mechanics.

To understand quantum computing, it is important to understand four key quantum mechanics principles:

- **Superposition**

  A qubit itself isn't very useful. But it can place the quantum information it holds into a state of superposition, which represents a combination of all possible configurations of the qubit. Groups of qubits in superposition can create complex, multidimensional computational spaces. Complex problems can be represented in new ways in these spaces.

  When a quantum system is measured, its state collapses from a superposition of possibilities into a binary state, which can be registered like binary code as either a zero or a one.

- **Entanglement**

  Entanglement is the ability of qubits to correlate their state with other qubits. Entangled systems are so intrinsically linked that when quantum processors measure a single entangled qubit, they can immediately determine information about other qubits in the entangled system.

- **Interference**

  Interference is the engine of quantum computing. An environment of qubits placed into a state of collective superposition structures information in a way that looks like waves, with amplitudes associated with each outcome.

  These amplitudes become the probabilities of the outcomes of a measurement

of the system. These waves can build on each other when many of them peak at a particular outcome or cancel each other out when peaks and troughs interact. Amplifying a probability or cancelling out others are both forms of interference.

- **Decoherence**
  Decoherence is the process in which a system in a quantum state collapses into a nonquantum state. It can be intentionally triggered by measuring a quantum system or by other environmental factors (sometimes these factors trigger it unintentionally). Generally, quantum computing requires avoiding and minimizing decoherence.

A computation on a quantum computer works by preparing a superposition of computational states. A quantum circuit, prepared by the user, uses operations to entangle qubits and generate interference patterns, as governed by a quantum algorithm. Many possible outcomes are cancelled out through interference, while others are amplified. The amplified outcomes are the solutions to the computation.

1.2. Quantum computing vs Classical Computing

Quantum computing is built on the principles of quantum mechanics, which describe how very small objects behave differently from large objects. But because quantum mechanics provides the foundational laws for our entire universe, on a very small level, every system is a quantum system. For this reason, we can say that while conventional computers are also built on top of quantum systems, they fail to take full advantage of the quantum-mechanical properties during their calculations. Quantum computers are expected to take better advantage of quantum mechanics to conduct calculations that even high-performance computers cannot. These machines generally perform calculations sequentially, storing data by using binary bits of information. Each bit represents either a 0 or 1. When combined into binary code and manipulated by using logic operations, we can use computers to create everything from simple operating systems to the most advanced supercomputing calculations.

Quantum computers, like classical computers, are problem-solving machines. But instead of bits, quantum computing uses qubits. Qubits are used to process data like traditional bits; however, by harnessing quantum phenomena, qubits have access to more complex mathematics for a different type of computation. This is due to quantum mechanical concepts known as superposition and interference, which were discussed earlier.

1.3. Quantum Circuits

Quantum algorithms are often diagrammatically represented as circuits in literature. In the circuit representation, qubits are represented by horizontal lines. Gates are then drawn on the qubits they act on. This is done in sequence from left to right. The initial state of the qubit is denoted at the beginning of each of the qubit lines. Notice that when we write down a mathematical expression for the circuit, the gates are written down from right to left in the order of their application.[1]

These principles are best illustrated by an example. Given in Fig. 1 is a circuit to preparing an entangled two qubit state called a Bell state from |00⟩

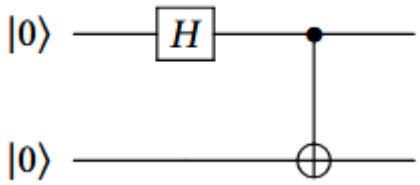


Figure 1. The circuit encoding the equation $CNOT_{12}(H \otimes I)|00 \geq \frac{1}{\sqrt{2}}$ (|00 > +|11 >)

We read the circuit from left to right. The qubits are numerically labelled starting from the top. First the H gate acts on the top most qubit changing the state of the system to

$$H \otimes I\,|00\rangle = (H\,|0\rangle) \otimes (I\,|0\rangle) = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle).$$

Then $CNOT_{12}$ acts on both of these qubits. The blackened dot on the first qubit implies that this qubit is the control qubit for the CNOT. The $\oplus$ symbol on the second qubit implies that this qubit is the target of the NOT gate (controlled by the state of the first qubit). The action of the CNOT then gives

$$\mathrm{CNOT}_{12}\left(\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)\right) = \frac{1}{\sqrt{2}}(\mathrm{CNOT}_{12}\,|00\rangle + \mathrm{CNOT}_{12}\,|10\rangle) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle).$$

The measurement of a qubit is also denoted by a special gate with a meter symbol on it, the presence of this gate on a qubit means that the qubit must be measured in the computational basis.

1.4. Quantum Algorithms

A quantum algorithm consists of three basic steps:

- Encoding of the data, which could be classical or quantum, into the state of a set of input qubits.
- A sequence of quantum gates applied to this set of input qubits.
- Measurements of one or more of the qubits at the end to obtain a classically interpretable result

  Below is the famous example of the Shor's Algorithm

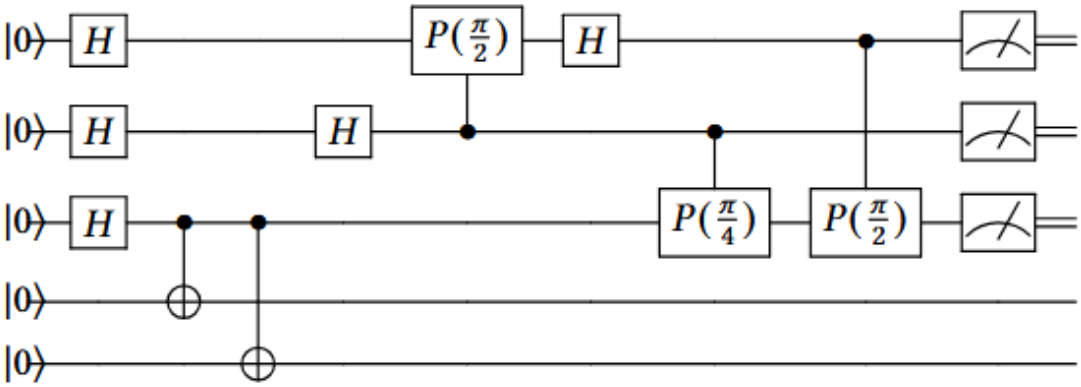


Figure 2. Shor's Algorithm for N=15 and x =11

First the encoding is done and Quantum gates are applied to a set of input qubits |0>to implement the algorithm.Measurement of Three qubits is done in the end we find the results of the said algorithm.

2. Neural Networks

A neural network takes an input vector of p variables $X = (X_1, X_2, \ldots, X_p)$ and builds a nonlinear function f(X) to predict the response Y.

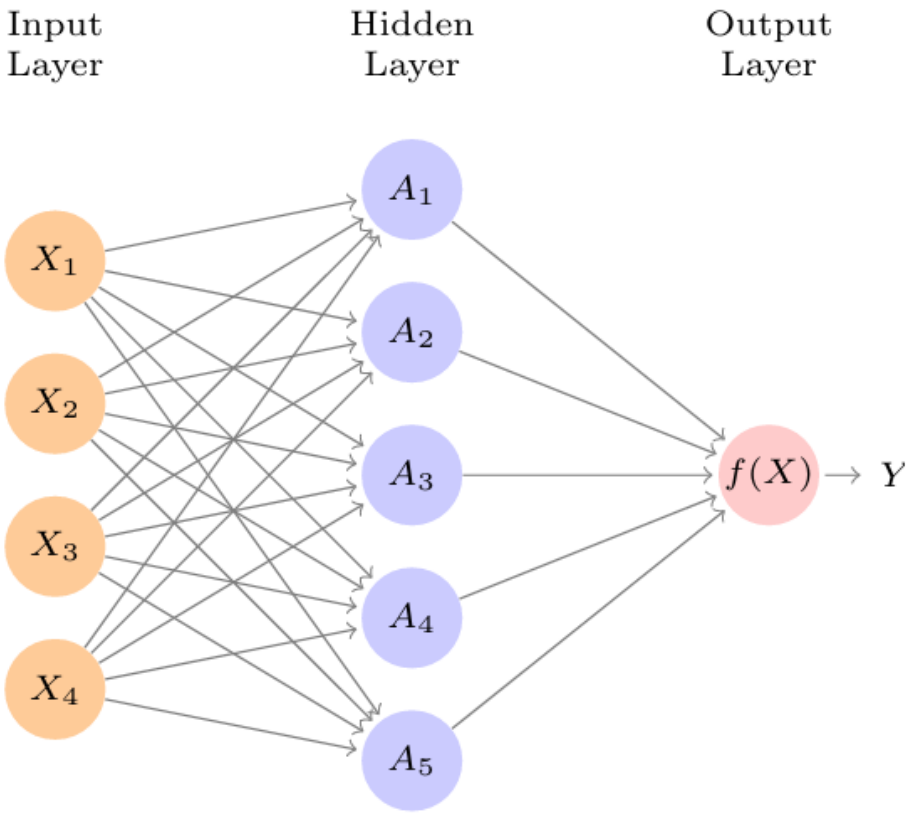


Figure 3. Single Layer Neural Networks [2]

Figure 3 shows a simple feed-forward neural network for modelling a quantitative response using p =4 predictors. In the terminology of neural networks, the four features X1,...,X4 make up the units in the input layer. The arrows indicate neural network that each of the inputs from the input layer feeds into each of the K hidden units (we get to pick K; here we chose 5). The neural network model has hidden units input layer the form

$$\begin{aligned} f(X) &= \beta_0 + \textstyle\sum_{k=1}^{K} \beta_k h_k(X) \\ &= \beta_0 + \textstyle\sum_{k=1}^{K} \beta_k g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j). \end{aligned}$$

It is built up here in two steps. First the K activations Ak, k=1,...,K,in activations the hidden layer are computed as functions of the input features $X_1, \ldots, X_p$

$$A_k = h_k(X) = g(w_{k0} + \textstyle\sum_{j=1}^{p} w_{kj} X_j),$$

where g(z) is a nonlinear activation function. In the early instances of neural networks, the sigmoid activation function was favoured.

$$g(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}},$$

which is the same function used in logistic regression to convert a linear function into probabilities between zero and one. The preferred choice in modern neural networks is the ReLU (rectified linear unit) activation function, which takes the form:

$$g(z) = (z)_+ = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise.} \end{cases}$$

A ReLU activation can be computed and stored more efficiently than a sigmoid activation. The name neural network originally derived from thinking of these hidden units as analogous to neurons in the brain.

2.1. Multiple Layer Neural Network

Modern neural networks typically have more than one hidden layer, and often many units per layer. In theory a single hidden layer with a large number of units has the ability to approximate most functions. However, the learning task of discovering a good solution is made much easier with multiple layers each of modest size. [2]
An example of Multiple Layer Neural Network is given with the help of MNIST handwritten digit problem in Figure 4

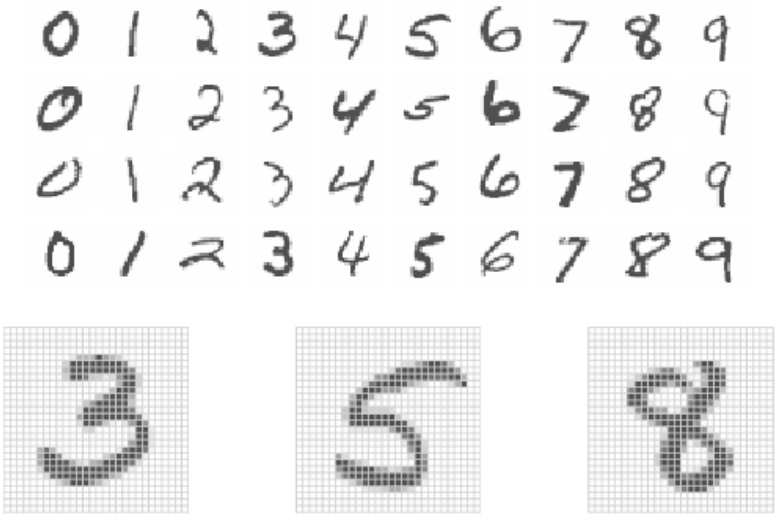


Figure 4. Examples of handwritten digits from the MNIST corpus. Each grayscale image has 28x28 pixels, each of which is an eight-bit number (0–255) which represents how dark that pixel is. The first 3, 5, and 8 are enlarged to show their 784 individual pixel values

Figure 5 shows a multilayer network architecture that works well for solving the digit-classification task. It differs from Figure 4 in several ways:

- It has two hidden layers L1 (256 units) and L2 (128 units) rather than one. Later we will see a network with seven hidden layers.
- It has ten output variables, rather than one. In this case the ten variables really represent a single qualitative variable and so are quite dependent. (We have indexed them by the digit class 0–9 rather than one-hot encoding 1–10, for clarity.) More generally, in multi-task learning one can predict different responses simultaneously with a single network; they all have a say in the formation of the hidden layers.
- The loss function used for training the network is tailored for the multiclass classification task.

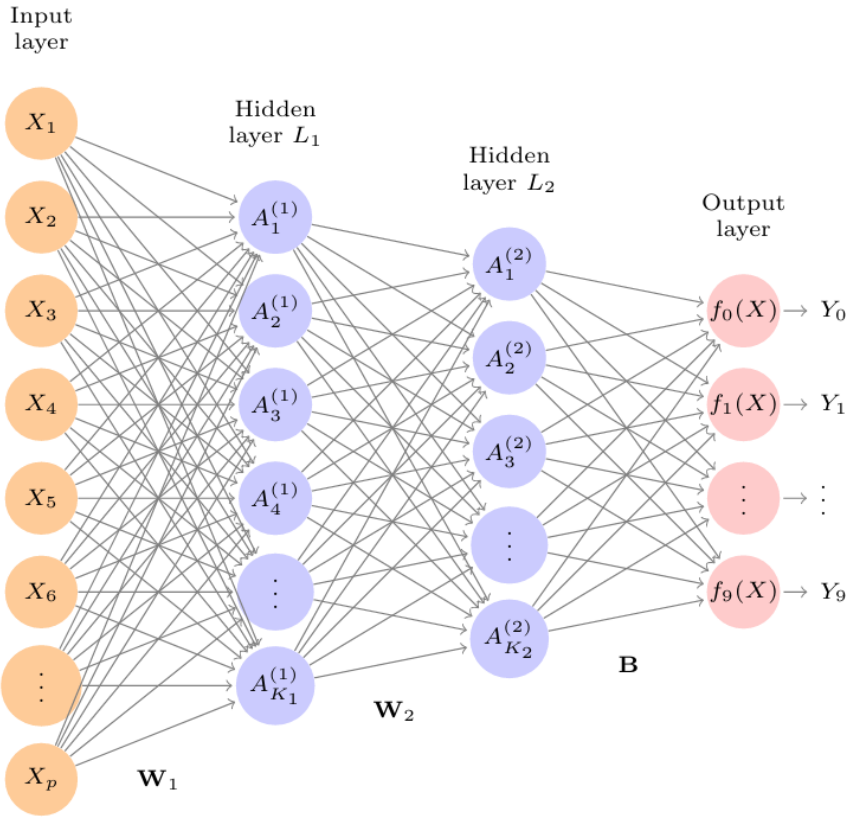


Figure 5. Neural network diagram with two hidden layers and multiple outputs, suitable for the MNIST handwritten-digit problem. The input layer has p =784 units, the two hidden layers $K_1 = 256$ and $K_2 = 128$ units respectively, and the output layer 10 units. Along with intercepts (referred to as biases in the deep-learning community) this network has 235,146 parameters (referred to as weights).

For representing class probabilities $f_{m(X)} = \Pr(Y = m|X)$, just like in multi bias nomial logistic regression, we use the special softmax softmax activation function

$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{\ell=0}^{9} e^{Z_\ell}},$$

The formula for cross entropy:

$$-\sum_{i=1}^{n}\sum_{m=0}^{9} y_{im} \log(f_m(x_i)),$$

3. Quantum Neural Network

Quantum neural networks are a sub-class of variational quantum algorithms that comprise quantum circuits containing parameterized gate operations. Information (usually in the form of classical data) is first encoded into a quantum state via a state-preparation routine called a quantum feature map.[3] The choice of feature map is geared towards enhancing the performance of the quantum neural network and is typically neither optimized nor trained. Once data are encoded into a quantum state, a model called a variational model is applied, which contains parameterized gate operations that are optimized for a particular task, analogous to classical machine learning techniques. The final output of the quantum neural network is extracted from measurements made to the quantum circuit after the variational model is applied. These measurements are often converted to labels or predictions through classical post-processing before being passed to a loss function, where the idea is to choose parameters for the variational model that minimize the loss function. We create two model variants: one which we call a quantum neural network and the other an easy quantum model.

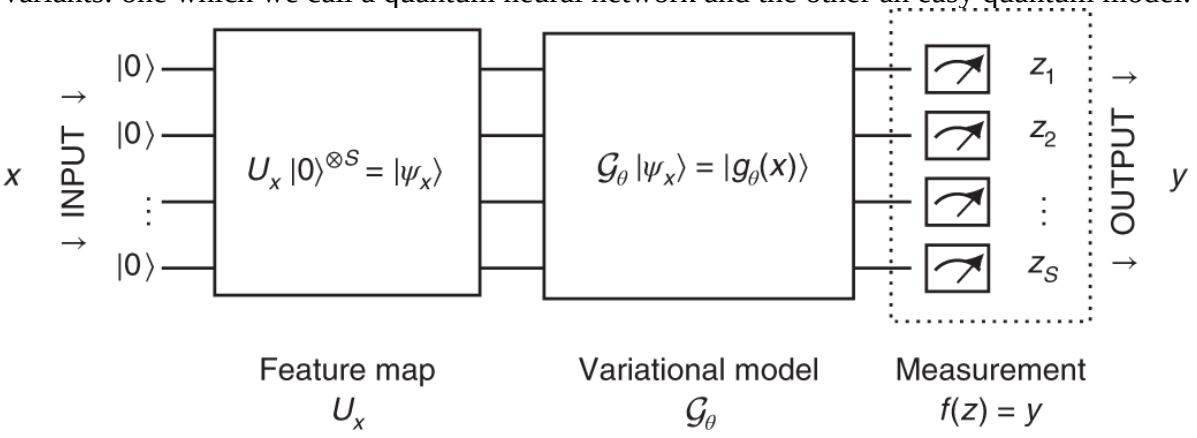


Figure 6. Overview of a Quantum Neural Network.

The input $x \in R^{\{s_{\{in\}}\}}$ is encoded into an S-qubit Hilbert space by applying the feature map $|\psi_x\rangle := U_x|0\rangle^{\{\otimes S\}}$. This state is then evolved via a variational form $|g_\theta(x)\rangle := G_\theta|\psi_x\rangle$, where G is a parameterized unitary evolving the state after the feature map to a new state, and the parameters $\theta \in \Theta$ are chosen to minimize a certain loss function. Finally a

measurement is performed whose outcome $z = (z_1, \ldots, z_S)$ is post-processed to extract the output of the model y:=f(z).

3.1.Feature Mapping

A quantum feature map is a function that transforms classical data into a quantum state within a Hilbert space. This process allows quantum computers to analyze data in a way that leverages the unique properties of quantum mechanics, such as superposition and entanglement, potentially leading to enhanced machine learning capabilities[4]. Given below is the feature mapping of Quantum Neural Network

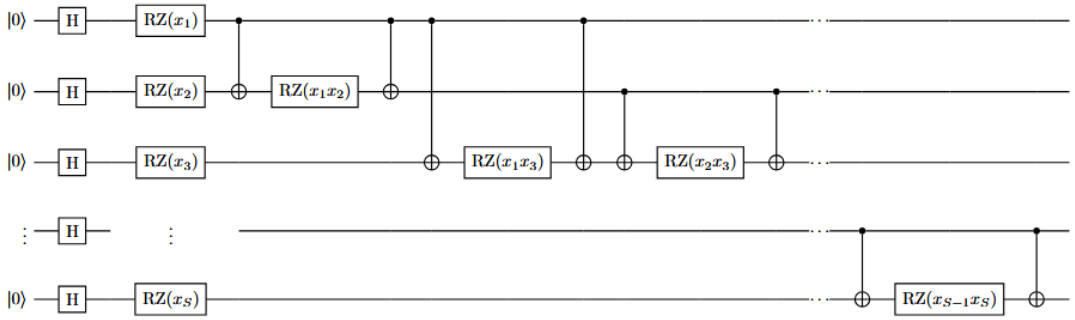


Figure 7. Feature Mapping of Quantum Neural Network

First, Hadamard gates are applied to each qubit. Then, normalized feature values of the data are encoded using RZ-gates. This is followed by CNOT-gates and higher order data encoding between every pair of qubits, and every pair of features in the data. The feature map is repeated to create a depth of 2. The easy quantum model, introduced in the results section, applies only the first sets of Hadamard and RZ-gates.[3]

3.2.Fisher Information Matrix

A way to assess the information gained by a particular parameterization of a statistical model is epitomized by the Fisher information. By defining a neural network as a statistical model, we can describe the joint relationship between data pairs (x, y) as $p(x, y; \theta) = p(y \mid x; \theta)p(x)$ for all $x \in X \subset R^{\{s_{\{in\}}\}}, y \in Y \subset R^{\{s_{\{out\}}\}}$ and $\theta \in \Theta \subset [-1,1]^d$ (where θ is a vectorized parameter set, Θ is the full parameter space and d is the number of trainable parameters). This is achieved by applying an appropriate post-processing function in both classical and quantum networks. In the classical network, we apply a softmax function to the final layer, whereas in the quantum network we obtain probabilities based on the parity of the output bit strings. The input distribution p(x) is a prior distribution, whereas the conditional distribution p(y|x;θ) describes the input–output

relation of the model for a fixed θ ∈ Θ; Θ forms a Riemannian space, which gives rise to a Riemannian metric, namely, the Fisher information matrix

$$\tilde{F}_k(\theta) = \frac{1}{k}\sum_{j=1}^{k} \frac{\partial}{\partial\theta}\log p(x_j, y_j;\theta)\frac{\partial}{\partial\theta}\log p(x_j, y_j;\theta)^{\mathrm{T}},$$

where $(x_j, y_j)_{j=1}^{k}$ are independent and identically distributed, drawn from the distribution $p(x, y; \theta)$ (ref. 37). By definition, the Fisher information matrix is positive semidefinite and hence its eigenvalues are non-negative, real numbers. The Fisher information conveniently helps capture the sensitivity of a neural network's output relative to movements in the parameter space[5]

3.3.Effective Dimension

The effective dimension is a complexity measure motivated by information geometry, with useful qualities. The goal of the effective dimension is to estimate the size that a model occupies in model space—the space of all possible functions for a particular model class, where the Fisher information matrix serves as the metric.

The effective dimension of a statistical model $M_\Theta := \{p(\cdot, \cdot; \theta): \theta \in \Theta\}$ with respect to $\gamma\epsilon(0,1]$ a d-dimensional parameter space $\Theta \subset R^d$and $n \in N, n > 1$ data samples is defined as

$$d_{\gamma,n}(\mathcal{M}_\Theta) := 2\frac{\log\left(\frac{1}{V_\Theta}\int_\Theta \sqrt{\det\left(\mathrm{id}_d + \frac{\gamma n}{2\pi\log n}\hat{F}(\theta)\right)}\,\mathrm{d}\theta\right)}{\log\left(\frac{\gamma n}{2\pi\log n}\right)},$$

where $V_\Theta := \int d\theta \in R_+$ is the volume of the parameter space. The matrix $\hat{F}(\theta) \in R^{\{d\times d\}}$is the normalized Fisher information matrix defined as

$$\hat{F}_{ij}(\theta) := d\frac{V_\Theta}{\int_\Theta \mathrm{tr}(F(\theta))\mathrm{d}\theta}F_{ij}(\theta).$$

In the limit n → ∞, the effective dimension converges to the maximal rank but it is worthwhile to note that the effective dimension does not necessarily increase monotonically with n.

4. Classical V/s Quantum Neural Network

4.1.Fisher Information Spectrum

Classically, the Fisher information spectrum reveals a lot about the optimization landscape of a model. The magnitude of the eigenvalues illustrates the curvature of a

model for a particular parameterization. If there is a large concentration of eigenvalues near zero, the optimization landscape will be predominantly flat and parameters become difficult to train with gradient-based methods

On the quantum side if a model is in a barren plateau, the Fisher information spectrum will be concentrated around zero and training also becomes unfeasible. We can thus make connections to trainability via the spectrum of the Fisher information matrix by using the effective dimension.

Then the cost function exhibits a barren plateau, the cost function gradient vanishes exponentially with the system size.[6] [7]

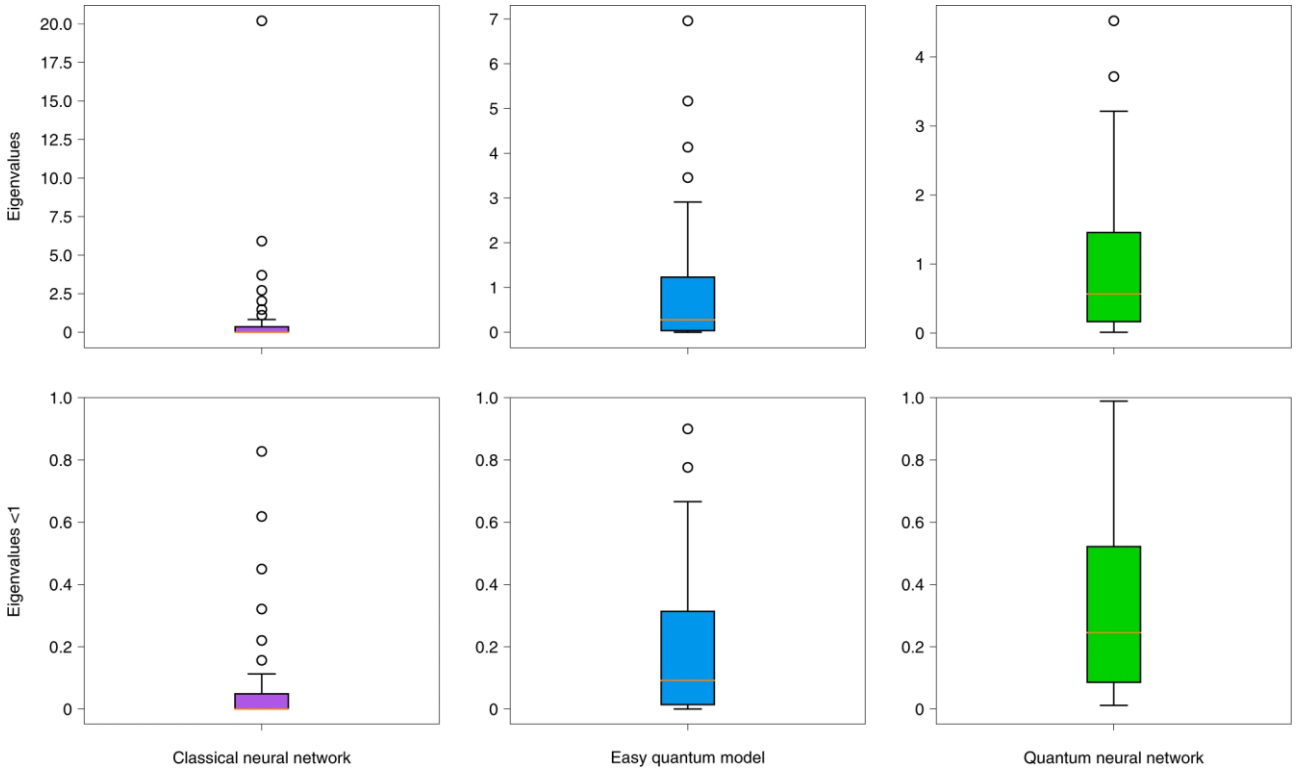


Figure 8. Box Plots of Average Eigenvalue Distribution of Fisher Information Matrix with Classical Neural Network And Two different quantum encodings[3]

The dots in the box plots represent outlier values relative to the length of the whiskers. The easy quantum model has a classically simulatable data encoding strategy, whereas the quantum neural network's encoding scheme is conjectured to be difficult. In each model, we compute the Fisher information matrix 100 times using parameters sampled uniformly at random and plot the resulting average distribution of the eigenvalues. We fix d = 40, input size at $s_{in} = 4$ and output size at $s_{out} = 2$. The top row contains the average distribution of all eigenvalues for each model, whereas the

bottom row contains the average distribution of eigenvalues less than 1 for each model.

4.2. Trainability

The observed Fisher information spectrum of the feedforward model is known to have undesirable optimization properties, where the outlying eigenvalues slow down training and loss convergence. These large eigenvalues become even more pronounced in bigger models as seen in previous figure.

On examining the easy quantum model over an increasing system size,the average Fisher spectrum becomes more concentrated around zero. This is characteristic of models encountering a barren plateau, presenting another unfavorable scenario for optimization. The quantum neural network, however, maintains its more even distribution of eigenvalues as the number of qubits and trainable parameters increase. Furthermore, a large amount of the eigenvalues are not near zero.

This highlights the importance of a feature map in a quantum model. The harder data encoding strategy used in the quantum neural network seems to structurally change the optimization landscape and remove the flatness, usually associated with suboptimal optimization conditions such as barren plateaus.

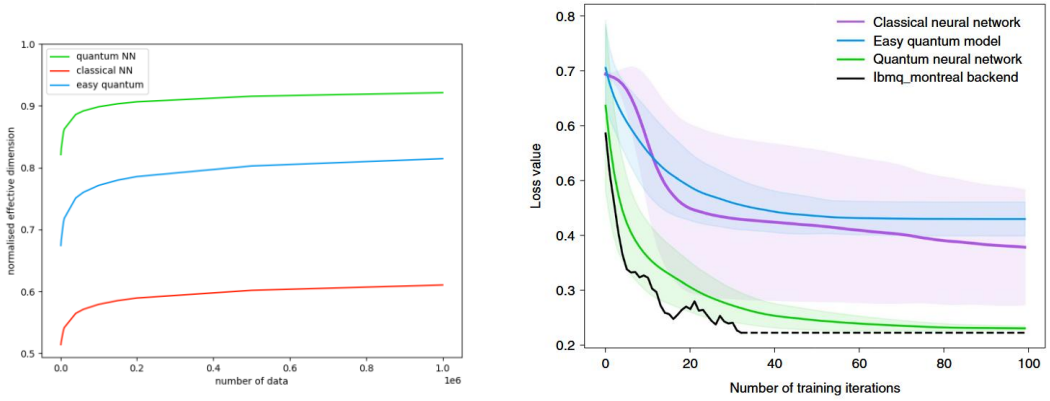


Figure 9. The normalized effective dimension plotted for the quantum neural network (green), the easy quantum model (blue) and the classical feedforward neural network (purple).

5. Conclusion

In stark contrast to classical models, understanding the capacity of quantum neural networks is not well explored. Moreover, classical neural networks are known to produce highly degenerate Fisher information matrices, which can considerably slow down training. No such analysis has been performed for quantum neural networks.

The feature map in a quantum model plays a large role in determining both its capacity

and trainability via the effective dimension and Fisher information spectrum.

Different variational circuits could also influence the model's landscape and the effects of non-unitary operations (for example, induced through intermediate measurements) should be investigated. The Fisher information spectra of certain quantum models seem robust against hardware noise, but trainability remains problematic and the possibility of noise-induced barren plateaus needs examination. Overall quantum neural networks can possess a desirable Fisher information spectrum that enables them to train faster and express more functions than comparable classical and quantum models—a promising reveal for quantum machine learning.

6. References

[1] A. J *et al.*, "Quantum Algorithm Implementations for Beginners," *ACM Transactions on Quantum Computing*, vol. 3, no. 4, pp. 1–92, Dec. 2022, doi: 10.1145/3517340.

[2] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, "Deep Learning," in *An Introduction to Statistical Learning*, in Springer Texts in Statistics. , Cham: Springer International Publishing, 2023, pp. 399–467. doi: 10.1007/978-3-031-38747-0_10.

[3] A. Abbas, D. Sutter, C. Zoufal, A. Lucchi, A. Figalli, and S. Woerner, "The power of quantum neural networks," *Nat Comput Sci*, vol. 1, no. 6, pp. 403–409, June 2021, doi: 10.1038/s43588-021-00084-1.

[4] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, Mar. 2019, doi: 10.1038/s41586-019-0980-2.

[5] T. Liang, T. Poggio, A. Rakhlin, and J. Stokes, "Fisher-Rao Metric, Geometry, and Complexity of Neural Networks".

[6] Y. Kwak, W. J. Yun, S. Jung, and J. Kim, "Quantum Neural Networks: Concepts, Applications, and Challenges," Aug. 02, 2021, *arXiv*: arXiv:2108.01468. doi: 10.48550/arXiv.2108.01468.

[7] H. Qi, L. Wang, H. Zhu, A. Gani, and C. Gong, "The barren plateaus of quantum neural networks: review, taxonomy and trends," *Quantum Inf Process*, vol. 22, no. 12, p. 435, Dec. 2023, doi: 10.1007/s11128-023-04188-7.